Fewer Tokens, Fewer Silent Errors: Output Format Design for Small Language Models in Real Time Order Parsing

Umar Ahmed
Independent Researcher
uakhmed@itmo.ru

Maxim Duanev
Independent Researcher
mduanev@itmo.ru

***Abstract*: Small language models are cheap to fine tune and fast to serve, but they hallucinate, and a hallucination that parses cleanly is invisible to the system that consumes it. We ask whether the *output format* can be designed to reduce that risk on a narrow task. We fine tune Qwen3.5-0.8B with LoRA on the public PIZZA semantic parsing benchmark using six lossless encodings of the same target tree, arranged as a ladder from 45.4 down to 11.9 tokens per utterance, with two matched token pairs that hold length fixed while varying surface form. Across 36 runs we find that shortening the output does *not* by itself improve accuracy: at 1000 training examples every ten tokens removed costs about four points of exact match. What matters is whether the surface form is made of natural words. A word based encoding matches the native s-expression on accuracy (0.830 versus 0.833, inside a measured rerun noise floor of 0.043) while using 3.8× fewer tokens, running 3.8× faster at batch size one, producing 4.4× fewer hallucinated entities, and halving the rate of errors that parse cleanly but are wrong. We argue this last quantity, which we call the *silent error rate*, is the right objective for deploying small models, and we report a café ordering system that runs at 80 ms per order on a consumer laptop.**

***Index Terms*: semantic parsing, small language models, hallucination, LoRA, output formats**

I. Introduction

A small language model is attractive for a narrow task. It fine tunes in minutes, runs on a laptop, and costs almost nothing per call. The objection is reliability: small models hallucinate, and in a structured output setting a hallucination is not an obviously broken string. It is a well formed tree describing an order nobody placed.

This distinction matters more than overall accuracy. An output that fails to parse is a *detectable* failure: the caller sees it, rejects it, and retries. An output that parses but is wrong is a *silent* failure: the caller has no signal, and a customer receives the wrong order. We therefore separate the two and treat the silent rate as the quantity to minimise.

Our question is whether the target format itself is a lever. The same semantic tree can be written many ways, and the choice changes how many tokens the model must emit, how fast it responds, and apparently how often it invents content. Our initial hypothesis was simple: fewer output tokens should mean less room to hallucinate, faster responses, and cheaper fine tuning.

That hypothesis is not supported in the form we posed it. Shortening the output alone does not improve accuracy, and at low data it clearly hurts. But a sharper version survives: shortening helps when the surface form stays close to natural language, and hurts when it becomes symbolic. Section V shows this with matched token controls that separate length from form.

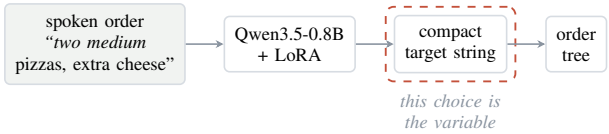


Fig. 1: The pipeline. Everything is held fixed except the format the model is trained to emit.

Contributions. (1) A ladder of six lossless encodings of the PIZZA target tree spanning 45.4 to 11.9 tokens, with matched token pairs that isolate length from surface form. (2) Evidence that length alone is not the mechanism, from twelve paired comparisons that all separate. (3) The silent error rate as a deployment objective, with a word based encoding halving it relative to the native format. (4) A measured rerun noise floor of 0.043 exact match, which bounds what any three seed comparison on this task can claim.

II. Task and System

A. The café parser

The work began with a deployed system. A café ordering booth must turn a spoken order, already transcribed, into a structured ticket. Orders arrive in Russian and English, with diminutives, self corrections and off menu items. We fine tuned Qwen3.5-0.8B with LoRA on a synthetic generator covering ten drinks, three sizes, five milks and three syrups, with a compact target format:

```
cap L oat;lat x2
```

where each item is `drink [size] [milk] [syrup] [xN]` and `?` marks an utterance that is not an order. On a hand written 49 case evaluation set, written separately from the generator and containing slang the generator never produces, the model reaches 48 of 49, emitting about three output tokens per order with a median latency of 80 ms. That figure is measured on an Apple M1 through a local Ollama server using the same

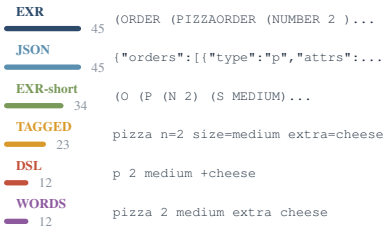


Fig. 2: The six encodings, all lossless and all preserving the dataset's own sibling order. EXR/JSON and DSL/WORDS are matched token pairs: nearly equal length, different surface form.

Q8_0 GGUF, which is 795 MiB on disk. The compact format uses roughly three times fewer output tokens than an equivalent JSON encoding, which is most of that latency.

This system works, but it cannot support a claim. The evaluation set is small, and the same author wrote both the generator and the test cases. To say anything general we moved to a public benchmark.

B. PIZZA

PIZZA [1] is a semantic parsing dataset of spoken pizza and drink orders paired with executable target trees. We use the public 10% training pool of 245,645 utterances, the 348 utterance development split and the 1357 utterance test split, of which 434 are marked as failures of the authors' PCFG baseline. Targets look like:

```
(ORDER (PIZZAORDER (NUMBER 2 )
(SIZE MEDIUM ) (COMPLEX_TOPPING
(QUANTITY EXTRA ) (TOPPING CHEESE ) ) ) )
```

We verified that no development or test utterance appears in the training pool, exactly or after normalisation, at any training size or seed.

III. Output Format Ladder

Comparing unrelated formats confounds length with surface form. We instead built six encodings of the *identical* tree that round trip to byte identical targets, four forming a length ladder and two held at matched length.

EXR is the dataset's native s-expression. JSON is a conventional structured encoding at nearly identical token count. EXR-SHORT replaces constructor names with single letters. TAGGED is flat `key=value` pairs. DSL is a compact grammar with sigils, where `+` marks an extra quantity and `!` a negation. WORDS shares the DSL's structure but spells markers out as ordinary words.

All six decode back to the identical tree on every development and test utterance and on a 50,000 example training sample, checked against the dataset's own matcher.

a) A tokenizer floor.: We also built a seventh encoding using short opaque entity codes, for example `p !p5 2 m3 +c2`. It has 16.4 characters per utterance against the DSL's 34.9, less than half, but *more* tokens, 11.9 against 10.8, because codes outside the subword vocabulary fragment. Shortening the string stops reducing tokens well before the information content does. This is why no arm sits below roughly ten tokens.

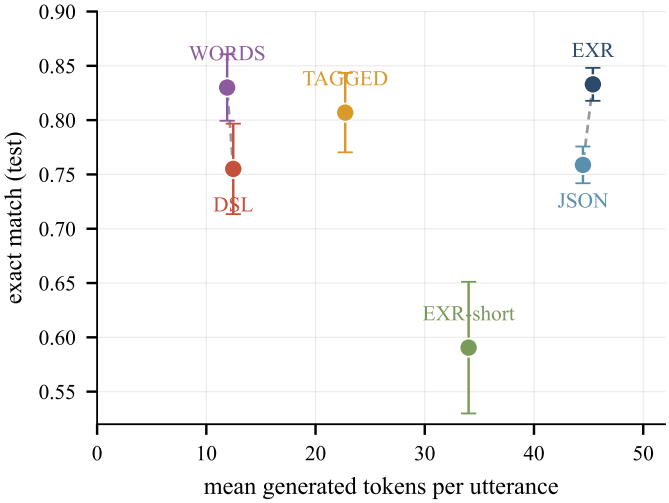


Fig. 3: Accuracy against output length. Dashed lines join the matched token pairs: within each pair length is held nearly constant, yet accuracy differs. Length alone does not determine accuracy.

IV. Experimental Setup

Identical across all arms: Qwen3.5-0.8B, LoRA with r=16, α=16 on all attention and MLP projections, batch 16, cosine schedule, loss on the completion only. The prompt is the same template the café model uses. Learning rate is selected per arm from $\{1e{-}4, 2e{-}4, 5e{-}4\}$ on development exact match at n=20,000 across three seeds, then frozen. Training uses early stopping on development exact match with patience 10, evaluated every 100 steps, to a maximum of 4000 steps.

`max_new_tokens` is set per arm to the longest target across all splits times 1.25, so no gold length output can be truncated. We log the share of generations reaching that cap; it is zero for every reported run.

Development drives every decision. Test is evaluated once per run and never read by any selection step. Exact match always comes from the dataset's own unordered matcher, which we never reimplement.

a) Noise floor.: We retrained one configuration twice under identical settings. Test exact match differed by 0.043. Any difference smaller than this is indistinguishable from rerun variance, and we do not interpret such differences.

V. Results

A. Shorter is not automatically better

Table I and Figure 3 show no monotone benefit from shortening. Regressing exact match on mean generated tokens across the four ladder arms gives a *positive* slope at both training sizes: at n=1000, removing ten tokens costs about four points of exact match. EXR-SHORT, the most aggressive structural compression, is the worst arm at 0.591, with parse validity of only 0.764. Our original hypothesis, stated as "fewer output tokens is better," is falsified.

TABLE I: Test results at n=20,000, mean of three seeds. Silent error is the share of utterances that parse cleanly but do not match the gold tree: the caller cannot detect these. Latency is median at batch size one on a shared GPU, and memory is this process's own GPU allocation during that measurement.

Format	Tokens	Exact match	Parse validity	**Silent error**	Halluc./utt.	Latency (ms)	Mem. (MiB)
EXR	45.4	0.833 ± 0.015	0.996	0.163	0.158	1213	1966
JSON	44.5	0.759 ± 0.017	0.904	0.146	0.145	1222	1707
EXR-short	34.0	0.591 ± 0.061	0.764	0.174	0.169	927	1706
TAGGED	22.7	0.807 ± 0.037	0.947	0.140	0.132	651	1706
DSL	12.5	0.755 ± 0.042	0.894	0.139	0.109	346	1705
WORDS	11.9	0.830 ± 0.031	0.900	0.070	0.036	322	1705

TABLE II: Matched token pairs at n=20,000. Paired bootstrap over test utterances. Equal length, different surface form, consistently different accuracy.

Pair	Seed	Tokens	Δ EM	95% CI	Verdict
EXR/JSON	0	45.3/44.9	+0.069	[+0.046, +0.091]	sep.
EXR/JSON	1	45.3/44.8	+0.088	[+0.069, +0.108]	sep.
EXR/JSON	2	45.6/43.7	+0.066	[+0.043, +0.090]	sep.
DSL/WORDS	0	12.5/12.1	-0.077	[-0.099, -0.057]	sep.
DSL/WORDS	1	12.5/11.7	-0.046	[-0.069, -0.023]	sep.
DSL/WORDS	2	12.4/12.0	-0.101	[-0.127, -0.074]	sep.

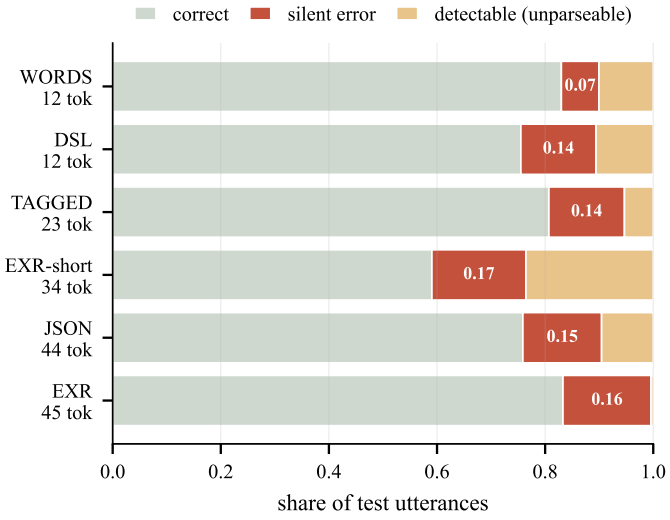


Fig. 4: Where the test set goes for each format, ordered by output length. Silent errors parse cleanly but are wrong, so a downstream system cannot detect them. Detectable failures can be rejected and retried.

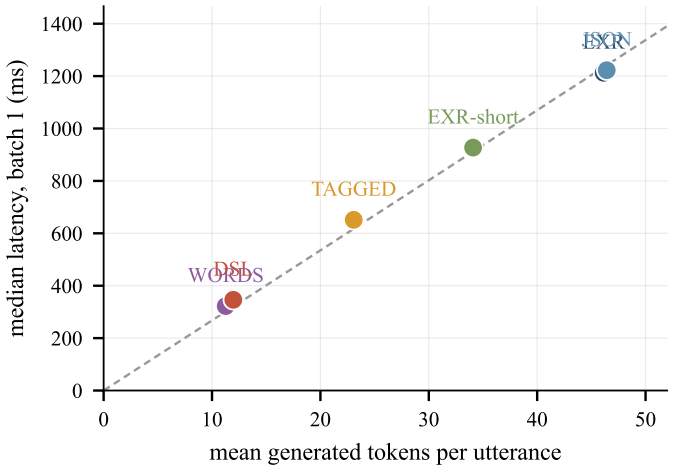


Fig. 5: Median latency at batch size one is close to linear in generated tokens. Throughput in tokens per second is flat across formats, so the speedup comes from emitting less, not from running faster.

B. Surface form, not length

The matched token pairs separate the two explanations, and all twelve comparisons separate under a paired bootstrap over the 1357 test utterances.

DSL and WORDS differ by less than a token in length, yet WORDS wins every seed. The two encode the same structure; they differ only in whether a quantity marker is written `+cheese` or `extra cheese`. Symbolic surface forms cost accuracy that natural word forms do not.

C. Silent errors

Figure 4 is the deployment picture. EXR has the highest parse validity at 0.996, but that is not an unalloyed good: nearly everything it gets wrong, it gets wrong *silently*, at 0.163. WORDS parses less often, 0.900, but its silent rate is 0.070, a factor of 2.3 lower. Its errors surface as parse failures a caller can catch.

The hallucination counts move the same way: 0.036 invented entities per utterance against 0.158 for EXR, a factor of 4.4, and 0.074 missing entities against 0.257.

D. Latency

Latency is close to proportional to output length (Figure 5). WORDS responds in 322 ms against 1213 ms for EXR, a 3.8× speedup, on a shared GPU under contention from other tenants. Tokens per second is flat across arms, confirming the gain comes from length rather than from a faster model.

Memory tells a different story. Across all eighteen measurements the process holds between 1705 and 1966 MiB, a spread of about fifteen percent that tracks the longer key value cache the verbose formats need. Weights dominate, and those are identical across arms. **Format choice buys latency and accuracy, not memory.** A practitioner picking a format to fit a memory budget is picking the wrong lever; the lever for memory is quantisation, which is what the deployed client in Section V-F uses.

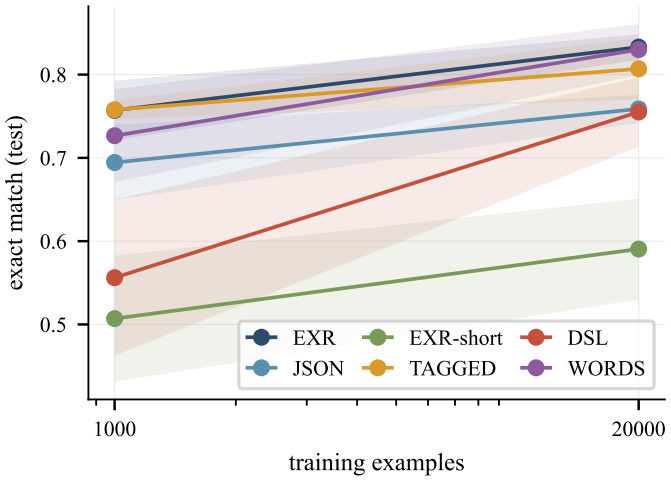


Fig. 6: Exact match against training set size. The penalty for symbolic formats is largest when data is scarce, and narrows as data grows.

E. Data efficiency

Figure 6 shows the format penalty is a low data phenomenon. At n=1000, DSL reaches 0.556 against WORDS at 0.727. By n=20,000 the gap narrows. A format closer to natural language appears to let the model reuse what pretraining already gave it, which matters most when there is little task data to learn from.

F. Deployment on device

The café parser runs entirely on an iPad, with no network call. The client is a SwiftUI application that links llama.cpp as an `xcframework` and offloads every layer to the Metal GPU. Table III lists what it costs to carry.

TABLE III: On-device footprint, measured on an iPad (A16) by the client's own benchmark: ten representative orders, three timed rounds after a discarded warm-up pass. The model is the café parser quantised to Q8_0.

Device	iPad15,7, iOS 26.3
Base model	Qwen3.5, 0.773 B parameters, 25 blocks
Quantisation	Q8_0, 8.63 bits per weight
Model file	795 MiB
Inference runtime	llama.cpp, 11 MiB framework (arm64)
GPU offload	all layers, Metal
Context window	512 tokens
Peak resident footprint	350 MiB
Model load time	2.5 s
Median latency	167 ms
p90 latency	354 ms
Throughput	16 tok/s

The spread between the median and the p90 is itself the paper's argument, restated on the hardware a customer waits in front of. Across the ten orders the benchmark parses, latency is close to linear in the number of output tokens: a least squares fit gives 48 ms of fixed overhead plus 52 ms per generated token ($R^2 = 0.91$), so a one token reply such as `esp` returns in about 100 ms while the longest, a seven token two drink order, takes about 414 ms. The format's token count is therefore not a throughput abstraction but the dominant term in the wait, and the 3.8× reduction the word based encoding buys becomes the same factor of responsiveness at the booth.

The footprint deserves a note, because 350 MiB is well under the 795 MiB model file. The figure is `phys_footprint`, which is what iOS enforces its per-process memory limit against, and llama.cpp maps the weight file rather than copying it, so the weight pages are file backed and are reclaimable under pressure instead of counting against that limit. The number to plan a deployment around is therefore the resident figure, not the file size, and on this device it leaves the parser a long way inside the budget of even an entry level iPad.

Two further details matter for reproducibility. First, the prompt template and the three stop strings compiled into the client are byte identical to those used for training and for every evaluation in this paper, so the deployed model sees exactly the input distribution it was fitted on. Second, the context window is 512 tokens, which the compact format fits comfortably and the equivalent JSON encoding would not at the longest orders. A format chosen for token count at training time therefore also decides whether the model fits a small on-device context at serving time.

The practical consequence is that a booth needs no server, no network round trip, and no per-request cost. The whole parser is a 795 MiB file beside an 11 MiB runtime.

G. Reproducibility

Every number in this paper is recomputed from saved model outputs rather than copied from a training log. Each of the 36 runs writes its configuration, package versions, training curves, adapter weights and the full prediction file for both evaluation splits. A prediction row carries the source utterance, the gold tree, the raw model output, the parsed tree or a null, a correctness flag and the generated token count, so exact match can be recomputed offline with no GPU. Every table and figure here is produced by two scripts that read only those files. The protocol, including the stopping rules and the statements this work declines to make, was written before any run and records its own later amendments.

VI. Discussion

a) Which format should a practitioner choose?: WORDS. It ties the native format on accuracy within the noise floor, and wins on every deployment axis: 3.8× fewer tokens, 3.8× lower latency, 4.4× fewer hallucinations, and 2.3× fewer silent errors. The recipe is to compress the *structure* while keeping the *vocabulary* natural.

b) Can a small model do this at all?: At 20,000 training examples a 0.8B model reaches 0.833 exact match. The published BART-Large result on this benchmark is 0.786, but it used 2.46M training examples, 123 times more. We do not claim to beat it. The useful statement is that comparable accuracy is reachable at a small fraction of the data and at a model size that serves in real time on consumer hardware, which the café system in Section II-A does at 80 ms per order.

c) Limitations.: One task, one model, one tokenizer. The tokenizer floor is specific to this subword vocabulary. Three seeds per cell gives wide intervals, which is why arm comparisons use a paired bootstrap over utterances rather than seed means. Learning rate is tuned at $n{=}20{,}000$ and reused at $n{=}1000$. Latency is measured on a contended shared GPU, so absolute milliseconds are inflated, though every arm meets the same contention. We have not tested whether these findings transfer to a larger model, where the format penalty may vanish.

VII. Conclusion

Output format is a free design variable that changes hallucination rate and latency by factors of three to four. The intuition that fewer tokens is safer is not quite right. What helps is compressing structure while keeping the surface in natural words, and what matters for deployment is not overall accuracy but how often a model is wrong in a way nobody can detect.

References

[1] K. Arkoudas *et al.*, "PIZZA: A new benchmark for complex end-to-end task-oriented parsing," 2022. Dataset: https://github.com/amazon-science/pizza-semantic-parsing-dataset